

Chapter 1: Data Structures: Trees & Balanced Search Trees

1.1 Hierarchical Tree Fundamentals

Nodes, Edges, Heights, and Depths

A tree is a non-linear, hierarchical data structure consisting of nodes connected by directed edges, containing exactly one root node and no cycles. The height of a node is the length of the longest path from that node down to a leaf. The depth of a node is the length of the path from the root to that node.

A Binary Tree is a tree where each node has at most two children, designated as left child and right child.

Key Formula / Law: Tree Math Properties: Max nodes at level $L = 2^L$ | Max nodes in binary tree of height $H = 2^{(H+1)} - 1$ | Min height for N nodes = $\lceil \log_2 N \rceil$

1.2 Binary Search Trees (BST)

The BST Invariant and Search Guarantees

A Binary Search Tree enforces the BST invariant: For every node X , all values in X 's left subtree are strictly less than $X.val$ (value $< X.val$), and all values in X 's right subtree are strictly greater than $X.val$ (value $> X.val$).

This invariant enables logarithmic search time $O(\log N)$ on average by discarding half the remaining search space at every step. However, if keys are inserted in sorted order, the BST degenerates into an unbalanced linked list with worst-case search complexity $O(N)$.

1.3 Tree Traversals

Depth-First and Breadth-First Exploration

Traversals systematically visit each node in the tree exactly once: 1. In-order Traversal (Left $\rightarrow$ Root $\rightarrow$ Right): Crucially visits nodes in strictly ascending sorted order for any valid BST. 2. Pre-order Traversal (Root $\rightarrow$ Left $\rightarrow$ Right): Useful for serializing or cloning tree topologies. 3. Post-order Traversal (Left $\rightarrow$ Right $\rightarrow$ Root): Crucial for bottom-up computation, deleting trees, or calculating subtree sizes. 4. Level-order Traversal (BFS): Explores nodes level-by-level using a FIFO Queue.

1.4 Self-Balancing Trees: AVL & Rotations

Maintaining Guaranteed $O(\log N)$ Height

An AVL tree is a self-balancing binary search tree where the balance factor of every node ($\text{height}(\text{left}) - \text{height}(\text{right})$) is restricted to $\{-1, 0, +1\}$. When an insertion or deletion violates this constraint, local tree rotations (Left Rotation, Right Rotation, Left-Right, Right-Left) restore balance in $O(1)$ time without violating the BST invariant.

Key Takeaways:

- A balanced BST guarantees $O(\log N)$ search, insertion, and deletion time.
- In-order traversal of a BST always yields keys in ascending sorted order.
- Unbalanced trees risk worst-case $O(N)$ degradation into linear structures.
- AVL trees maintain logarithmic height by executing $O(1)$ pointer rotations upon imbalance.